Evolution of deployment technologies

Era	Technology	Key features	Pros	Limitations
Early 2000s	Virtual machines (VMs)	Full OS per instance	Isolation, multi-tenancy	Heavy, slow to start
2013	Containers (Docker)	OS-level virtualisation	Lightweight, fast, portable	Needs orchestration
2014	Kubernetes	Container orchestration	Auto-scaling, self-healing	Complex setup, infra overhead
2014	Serverless functions (e.g., AWS Lambda)	Event-driven code	No servers, pay-per-use	Short-lived, limited control
2018+	Serverless containers	Containers + serverless infrastructure	Full apps, no infra management	Cold start latency, vendor lock-in

Containers came next. Containers virtualise the operating system rather than the hardware, as VMs do. They are considerably lighter as a result. Code, libraries and configuration are all included in a container. It functions the same everywhere because everything is packaged together. The issue that developers frequently encountered — "It works on my machine, but not on yours" — was addressed by this.

With its 2013 launch, Docker significantly accelerated the rise in popularity of containers. It simplified the process of building, shipping and operating containers. Then, in 2014, Google unveiled Kubernetes, an open source tool for managing numerous containers on various computers. Scheduling, scaling, restarting failed containers, and other aspects of container management were automated by Kubernetes.

However, even with Kubernetes, managing containers became more complex as applications expanded and more were developed. Cluster configuration, update management, and traffic pattern monitoring required time from developers. It distracted attention from the important tasks of creating quality software and enhancing user experience.

Serverless computing was developed to reduce this load. Developers create code serverless and upload it to the cloud. Only when necessary does the cloud run it. No servers to manage. No planning for capacity. One of the first significant platforms to offer this model was AWS Lambda, which launched in 2014. Azure Functions and Google Cloud Functions quickly followed.

However, serverless functions like Lambda are limited. They're good for small, short tasks triggered by events, like resizing an image or responding to a web request. But they're not ideal for full applications or long-running processes.

That's where serverless containers come in. They combine the portability and power of containers with the ease and efficiency of serverless computing. They're flexible, scalable and cost-effective, making them a compelling choice for modern cloud-native development.

What are serverless containers?

Essentially, a serverless container is like a standard container. After building your application, you package it into a container image along with all its necessary code, libraries and dependencies. How and where you run it is what differs.

Traditional containers must be deployed on a Kubernetes cluster or server. You have to choose how much processing power to allocate, set up networks and prepare for spikes in traffic. You don't need to worry about any of that with serverless containers.

You upload your container image to the cloud when you use a serverless container platform. That's all. When a user opens your website or uploads a file, for example, the platform automatically starts a container to process the request. The container shuts down when demand declines. You are not billed if there are no requests.

There are numerous reasons why this model is appealing. Infrastructure management, scaling concerns, and idle resource payments are all removed. Additionally, serverless containers can run background jobs, microservices, or full-fledged applications, in contrast to serverless functions, which frequently have resource and execution time restrictions.

Consider a photo processing app, for instance. A server would need to be always running in the conventional model in case a user uploaded an image. However, when using serverless containers the application starts only when a photo is uploaded. Your expenses will stop once the processing is finished.

Additionally, compared to serverless functions, serverless containers offer greater flexibility. You can use any programming language, create your own environment, and add all the tools your app requires. With serverless functions, you are not constrained by the memory caps or particular formats.

Key platforms and tools

Serverless container services are provided by several cloud providers. The fundamental concept is the same: